

/*
Problema 3 En esta práctica se pretende analizar cómo se representa un número en coma flotante de precisión simple (formato IEEE 754 de 32 bits) en memoria.

Se pide desarrollar un programa en lenguaje C que:

1. Declare una variable de tipo float que se lepedirá al usuario (se puede probar por ejemplo con el valor -36).
2. Muestre en pantalla:
 - o El valor en formato decimal estándar.
 - o El valor en formato hexadecimal científico utilizando el especificador %a.
3. Acceda a la representación interna del número en memoria (sus 32 bits) utilizando una union que permita interpretar los mismos bits como:
 - o float
 - o uint32_t
4. Muestre:
 - o La representación en memoria en formato hexadecimal.
 - o La representación completa en binario (32 bits).
5. En la salida binaria:
 - o Separar el bit de signo.
 - o Separar los 8 bits del exponente.
 - o Separar los 23 bits de la mantisa.

Requisitos técnicos

- Utilizar las librerías <stdio.h> y <stdint.h>.
- No utilizar funciones externas para convertir a binario.
- Generar manualmente la representación bit a bit usando desplazamientos (>>) y operador AND (&).
- El programa debe compilar sin errores y ejecutarse correctamente.

Ejemplo de salida con valor -36
Valor decimal: -36.000000
Hexadecimal: -0x1.200000p+5
Representación en memoria (hex): 0xC2100000
Bits: 1 10000100 0010000000000000000000
*/

```
#include <stdio.h>
#include <stdint.h>

typedef union {
    float f; //float
    uint32_t b; //bits
} FloatBits_u;

int main(int argc, char* argv[]){

    //1 Recogemos el numero que introduzca el usuario.
    float numeroUsuario;

    printf("Bienvenido. Introduzca un numero: \n");
    scanf("%f", &numeroUsuario);

    //2 Mostramos por consola el numero transformado al formato requerido
    printf("El numero introducido en formato decimal estandar es: %f \n", numeroUsuario);
    printf("El numero introducido en formato hexadecimal científico estandar es: %a \n", numeroUsuario);

    //3 y 4 Accedemos a la representación interna del número en memoria (sus 32 bits)
    FloatBits_u floatBits;
    floatBits.f = numeroUsuario; //Escribimos el float y ahora sigue teniendo los mismos 32 bits pero interpretados como un entero sin
signo

    printf("Representacion en memoria (hex): 0x%X \n", floatBits.b);

    printf("Representacion en memoria (bits): \n");
    for (int i = 31; i >= 0; i--) {
        uint32_t bit = (floatBits.b >> i) & 1; // WARNING: Para la realización de esta parte, me he ayudado de las indicaciones de la IA Claude
        printf("%u", bit);

        if(i == 31 || i == 23){ //5 Separamos en partes la salida binaria
            printf(" ");
        }
    }
    printf("\n");
    return 0;
}
```